Mastering SentinelFlow: study plan

Private file (gitignored). Goal: you can explain every part of SentinelFlow without notes, answer the hard
questions an interviewer will ask, and change the code yourself. About 3 weeks at ~2 hours a day.

Rules for every phase

1. **Read** the file with the questions below in mind. Don't skim: you should be able to say what each function is for.
2. **Run** it and look at the real output.
3. **Break** something on purpose, then see which test fails and why. Undo with `git checkout -- <file>`.
4. **Explain out loud** (or in writing, in your own words) before you tick the box. If you can't explain it, you don't know it yet.

Keep a file `my_answers.md` next to this one and write your answers there. Interview prep is just re-reading that file.

Line numbers below are from 2026-10-30 and will drift; search for the function name if they don't match.

Phase 1 — The big picture (days 1–2)

Read: `README.md`, then `sentinel-flow/docs/GUIDE.md` sections 1–3 (what it does, network basics, how the pieces fit).

Run:

```
cd ~/ExtraSpace/cv_work/sentinel-flow
make test-sensor && make test-engine
# terminal 1
engine-ml/.venv-runtime/bin/python engine-ml/daemon.py --db /tmp/sf.db --
jsonl /tmp/sf.jsonl --mitigate dry-run
# terminal 2
sensor-c/build/sensor -r pcaps/test_tls_full.pcap -v
python3 engine-ml/query_dashboard.py --db /tmp/sf.db
```

Check yourself

- [ ] Draw the architecture on paper from memory: the 3 planes, what goes over each arrow (format + transport), where data is stored.
- [ ] Say in 60 seconds what SentinelFlow does and why it doesn't need to decrypt TLS.
- [ ] Name every technology in the stack and say in one sentence why it's there (GUIDE section 6 has the list).

Phase 2 — Networking and the C sensor (days 3–6)

Technologies to learn first

Technology	What you must know	Where to learn
TCP/IP, flows	IPv4 header, TCP/UDP ports, the 5-tuple, what a "flow" is, TCP handshake	GUIDE §2; any networking course chapter on transport layer
libpcap	`pcap_open_live` / `pcap_open_offline`, `pcap_dispatch`, link types (DLT_EN10MB, SLL, SLL2), BPF filters	tcpdump.org → "Programming with pcap" tutorial
nDPI	What deep packet inspection is, protocol detection from the first packets, risk flags, why detection "gives up"	github.com/ntop/nDPI README + `example/ndpiReader.c`
JA4	How a TLS ClientHello becomes a fingerprint, the parts `a_b_c`	github.com/FoxIO-LLC/ja4 (JA4 spec)
uthash	`HASH_FIND`, `HASH_ADD`, `HASH_DEL`, iterating with `HASH_ITER`	troydhanson.github.io/uthash/userguide.htr
Welford's algorithm	Running mean/variance in one pass without storing values; sample vs population variance	Wikipedia: "Algorithms for calculating variance"
ZeroMQ	PUSH/PULL pattern, high-water mark, what happens when the receiver is slow	zguide.zeromq.org chapters 1–2
Sanitizers	What ASan, UBSan and LeakSanitizer catch, and how to build with `-`	clang/gcc docs; REPORT Part 3 §3.1 for how it was done here

Technology	What you must know	Where to learn
	`fsanitize=address,undefined -g`	
CMake	How `sensor-c/CMakeLists.txt` finds nDPI (`NDPI_ROOT`)	the file itself

Files, in reading order

1. **`sensor-c/src/stats.h`** (73 lines): `welford_init` , `welford_update` , `welford_variance` .
   - [ ] Compute mean and sample stddev of {2,4,4,4,5,5,7,9} by hand step by step, and compare with `test_stats.c` case 2 (≈ 2.138).
   - [ ] Why one pass and not "store all inter-arrival times"? (memory per flow × thousands of flows)
2. **`sensor-c/src/flow_table.h` / `flow_table.c`** : the flow key and how both directions of one connection map to the same entry.
   - [ ] Explain the invariant tested in `src/test_flow.c` (the "larger IP initiates" case).
3. **`sensor-c/src/main.c`** (the core, ~780 lines):
   - `ipv4_offset` (~308): finding the IP header for each link type. Why the `rd16be` helper exists (the misaligned-read UBSan bug).
   - `packet_callback` (~357): per-packet path: parse, find/create flow, update counters and Welford stats, feed nDPI.
   - `emit_flow` (~121) and `export_flow` (~233): building the 23-field JSON, `record_hash` to avoid repeat records.
   - `expire_idle` (~274), `snapshot_if_due` / `take_snapshots` (~257–282): idle timeout (30 s) vs active timeout (`-A`).
   - `main` (~592): options, live vs offline, `poll` + `pcap_breakloop` (why a quiet interface used to hang the sensor).
4. **`sensor-c/src/ndpi_wrapper.c`** : `is_tls_family` (~114) and `is_valid_ja4` (~130). Why JA4 is only read for TLS flows (nDPI keeps per-protocol data in a C `union` : reading the TLS field of a DNS flow gave garbage).
5. **`sensor-c/src/zmq_publisher.c`** : `zmq_publisher_send` with a patience timeout, `zmq_publisher_destroy` drains at exit. Link this to the "1,001 of 1,105 delivered" silent-loss bug (README → RESULTS.md, engineering issues).
6. **Tests:** `src/test_flow.c` , `src/test_stats.c` , `tests/test_sensor_pcaps.py` (14 regression tests, each tied to a bug).

Exercises

- [ ] Run `sensor-c/build/sensor -r pcaps/test_tls_full.pcap -v` and explain **every one of the 23 fields** of one TLS record. Check them against `tshark -r pcaps/test_tls_full.pcap -q -z conv,tcp`.
- [ ] Break it: in `ndpi_wrapper.c`, make the JA4 read ignore `is_tls_family`. Rebuild (`make sensor`), run `make test-sensor`, and explain which test fails and why.
- [ ] Break it: make the IAT update in `main.c` run on the first packet too (it's guarded by `pkt_count > 1`). Does any test catch it? If not, write one in `tests/test_sensor_pcaps.py`.
- [ ] Build once with sanitizers and replay the 4 pcaps; you should see 0 errors.
- [ ] Read the "Engineering issues found and fixed" table in `docs/RESULTS.md`. For each bug, find the fix in the code and the test that guards it.

Questions you must be able to answer

- What is the difference between the idle timeout and the active timeout, and why does live capture use `-A 60`?
- Why is `ndpi_risk_mask` sent as an exact 64-bit integer (hint: JSON numbers and 2^53, test `test_risk_mask_is_exact_above_2_53`)?
- What happens to flows still open when you press Ctrl+C?
- Why libpcap and not raw sockets or AF_XDP? What would you change to go faster?
- Why is the sensor in C and the rest in Python?

Phase 3 — Daemon, storage, dashboards (days 7–8)

Technologies

Python `threading` + `queue`, input validation, SQLite (WAL mode), JSON Lines, ClickHouse (column store, HTTP interface), Grafana provisioning, Docker Compose.

Files

- **`engine-ml/daemon.py`**: `validate` (~101), `OnnxScorer` (~201: loading models, SHA-256 check, `_lgbm_norm` calibration), `alert` (~285: severe vs any risk policy), `finalize` / `enrich`, `SQLiteStore` (~328), `ClickHouseStore` (~382), `ZmqSinkTransport` (~481), `main` (~565: batching 1,000 rows or 1 s, `flush`, signal handling).
- `engine-ml/sql/init_clickhouse.sql`, `docker-compose.yml`, `dashboard/grafana/` (datasource + dashboard JSON).
- `engine-ml/query_dashboard.py`: skim; know what each of the 7 panels shows.

Exercises

- [ ] Replay stored flows without the sensor: `daemon.py --from-jsonl /tmp/sf.jsonl --db /tmp/sf2.db`.
- [ ] Open the SQLite DB yourself: `sqlite3 /tmp/sf.db ".schema"` and write one query: top 5 flows by `ml_anomaly_score`.
- [ ] `make stack-up`, run the daemon, open Grafana on :3000, find your flows. `make stack-down` after.
- [ ] Send the daemon a broken record (missing field) via `--from-jsonl` and find it in `rejected.jsonl`.

Questions

- Why batch writes instead of writing each flow?
- Why does the daemon fall back to a heuristic scorer, and why does that scorer never block?
- What does WAL mode give you, and why can the dashboard read while the daemon writes?
- Why ClickHouse and not PostgreSQL for this data?

Phase 4 — The machine-learning pipeline (days 9–12): expect the hardest questions here

Concepts to learn first

Concept	What you must be able to explain	Where to learn
Gradient-boosted trees (LightGBM)	How boosting builds trees on residuals; key params in `model_meta.json` (300 trees, lr 0.05, 31 leaves)	LightGBM docs "Features"; StatQuest videos on gradient boosting
IsolationForest	Unsupervised anomaly score; why it's kept but not used for alerts	scikit-learn user guide
Metrics	precision, recall, FPR, ROC-AUC vs PR-AUC (and why PR-AUC matters with imbalance)	scikit-learn "Model evaluation" page
Data leakage + grouped CV	Why flows from the same capture must not be in both train and test	scikit-learn `GroupKFold` docs

Concept	What you must be able to explain	Where to learn
Calibration	Mapping a score so that a threshold means a known false-alarm rate	GUIDE §5; `lgbm_calibration` in `train_pipeline.py`
Base-rate problem	1 % FPR × huge number of benign flows = many false alarms	search "base rate fallacy intrusion detection" (Axelsson 2000)
ONNX	Exporting the model so the daemon needs only `onnxruntime`, parity check	onnx.ai, `export_onnx` and `onnx_parity_and_latency`

Files, in order

1. **`engine-ml/features.py`**: `extract` (~302), `_ja4_parts` (~119), `is_internet` (~140), `ConversationTracker` (~200; `observe`, `_features`, `stats`). Read REPORT Part 9 alongside it.
2. **`engine-ml/models/build_dataset.py`**: `SensorRunner` (~88: the dataset goes through the *real* sensor), `mta_sources` (~163), `label_flow` (~197: how IOC lists turn into labels; benign / malicious / ambiguous). Then open `engine-ml/datasets/mta_labels.json`.
3. **`engine-ml/models/train_pipeline.py`**: `group_weights` (~70) and `train_weights` (~91: capping each capture's weight), `fit_fold` (~118), `cross_validate` (~177), `lgbm_calibration` (~101), `export_onnx` (~205), `onnx_parity_and_latency` (~223).
4. **`engine-ml/models/evaluate_holdout.py`**: how the 5 holdout captures go through the *live* daemon.
5. **Outputs:** `models/model_meta.json`, `models/evaluation.json` (per-fold numbers), `models/evaluation_holdout.json` (per-case numbers).

Exercises

- [ ] Open `evaluation.json`. Fold 1 recall is 0.93, folds 2–3 are 0.53–0.56. Find out **why** (which captures/families are in those folds; families seen in only one capture). Write the answer down: you will be asked this.
- [ ] Read one holdout case in `evaluation_holdout.json` (e.g. DarkGate) and explain every number.
- [ ] Work out the base rate yourself: a small office makes ~50,000 flows/hour. At 1 % FPR, how many false alerts per hour? Now explain why blocking needs the per-IP rule, the allowlist, and why the real-host test still gave 9–67 blocks/hour.
- [ ] (Colab, ~700 MB download) Retrain: commands in `docs/USAGE.md` §7. Then run `train_pipeline.py --no-export` with the 5 `conv_*` features removed and compare recall with the current model: you've reproduced the fv1 → fv2 gain yourself.

Questions

- Why are IP addresses and timestamps **not** features?
- Why grouped CV by capture? What would a random split have shown, and why would it be a lie?
- What does "0.80 = 1 % of held-out benign flows" mean, and how is it computed?
- How are labels made, and where are they wrong? (internal attacker traffic counted as benign; shared services marked ambiguous)
- Why is njRAT still missed even though its timing is a perfect beacon? (only raw-TCP beacon in training; the protocol features dominate)
- Why ONNX instead of pickling the model? (smaller runtime, no training libs in production, SHA-256 check, parity test 0 / 12,127 decisions differ)
- What would you do with 3 more months? (benign data from real hosts, bigger and newer holdout, beacon rule)

Phase 5 — Blocking: nftables, network namespaces, safety (days 13–14)

Technologies

netfilter / nftables (tables, chains, hooks, priorities, sets with `flags timeout`, counters), Linux network namespaces and veth pairs, IP spoofing.

Files

- **`engine-ml/mitigator.py`**: `WindowDistinct` (~111), `BlockPolicy` (~203): `why_protected` (~251), `_flow_decision` (~306), `_dns_server` (~296), `_scan_decisions` (~350), `_note_server` (~426). `NftablesBackend` (~506): read the ruleset in `setup` (~542) line by line. `Mitigator.process` / `_apply` / `refresh` (~605–694).
- **Tests:** `engine-ml/tests/test_mitigator.py` (38), `engine-ml/tests/lab_nftables_netns.py`, `tests/verify_closed_loop.sh`.

Exercises

- [ ] Fetch the DarkGate capture (`python3 engine-ml/datasets/fetch_mta.py --holdout --only 2023/11/20`) and run `make verify-closed-loop`. Read the script while it runs and explain each step.
- [ ] By hand: create two namespaces joined by a veth pair, ping between them, block one side with `python3 engine-ml/mitigator.py --netns <ns> block <ip>`, watch the ping fail

and the counter go up (`sudo ip netns exec <ns> nft list table inet sentinel`), then unblock.

- [ ] Explain the 3 chains: why `prerouting` checks `saddr` but `output` / `forward` check `daddr` .

Questions

- Why block per IP (one flow ≥ 0.95 or two ≥ 0.80 in an hour) instead of per flow?
- Why is a DNS resolver never a block target?
- How could an attacker abuse automatic blocking? (forge a "scan" from 8.8.8.8 so you block your own resolver.) What does the code do about it?
- Why do blocks expire after 15 minutes?
- Why nftables sets and not one iptables rule per IP?

Phase 6 — Make the XDP part yours (days 15–18): the biggest CV upgrade

You provided `mitigation-ebpf/xdp_blocker.bpf.c` and `xdp_ctl.c` . They compile, but attaching failed (REPORT Part 4 §6):

```
libbpf: map 'blocked_ips': attr 'type': expected PTR, got int.
```

Fixing this yourself makes the "kernel" part of the project true, and it's work you can explain in full because you did it.

Learn first

- eBPF basics: programs, the verifier, maps, BTF, pinning to `/sys/fs/bpf` . Book: *Learning eBPF* (Liz Rice), chapters 1–5.
- XDP: where the hook runs, `XDP_PASS` / `XDP_DROP` , generic vs native mode. Hands-on: github.com/xdp-project/xdp-tutorial (basic01–basic04, packet01–packet03).
- Tools: `bpftool prog` , `bpftool map` , `bpftool btf dump file <obj>` , `ip link set dev X xdpgeneric obj ...` .

Steps

1. [ ] Find the cause of the error. Compare the hand-written `SEC` , `__uint` , `__type` and helper definitions at the top of `xdp_blocker.bpf.c` with libbpf's `<bpf/bpf_helpers.h>`

(package `libbpf-dev`), and dump the object's BTF to see how the map's `type` field was encoded.

2. [ ] Fix it (using the real libbpf headers is the normal way), rebuild with `clang -O2 -g -target bpf -c`.
3. [ ] Make the loaded program and `xdp_ctl` use the **same** map. Check where the map actually gets pinned when you load it, and where `xdp_ctl` looks (`/sys/fs/bpf/sentinel_blocked_ips`).
4. [ ] Attach to a veth in a throwaway namespace (`xdpgeneric`), add an IP with `xdp_ctl add`, prove packets drop (ping + `xdp_ctl list` counters).
5. [ ] Add an `XdpBackend` to `mitigator.py` with the same methods as `NftablesBackend` (`setup`, `block`, `unblock`, `list`, `teardown`), plus `--mitigate xdp` in `daemon.py`, plus tests.
6. [ ] Think about this before an interviewer asks: XDP only sees **incoming** packets. What happens to outgoing C2 traffic? (Keep nftables for `output`, or add a TC egress program.) Also: the map has no timeout like the nftables set; who removes expired entries?
7. [ ] Measure drops/second with XDP vs nftables in namespaces (e.g. `hping3 --flood` or `pktgen`) and add the result to `docs/RESULTS.md`.
8. [ ] Update README "Limits" and "Roadmap" once it works. Commit in small steps with clear messages.

Phase 7 — Write up the live malware lab (days 19–20)

Your strongest story, and it isn't in the public docs yet. Per your notes (`HANDOFF.md`, 2026-10-29): 27 RAT samples (AsyncRAT, njRAT, RemcosRAT, Formbook) detonated in the isolated Win-Detonate VM, 12 reached a live C2, captures and indicators in `downloads/malware_lab/`.

- [ ] Decide which model is official: the committed `58fb9fc3`, or the retrained `634e3564` (backups in `downloads/malware_lab/model_fv2_*`).
- [ ] Replay each C2 capture through the sensor + daemon yourself and record: C2 alerted? blocked? top score? false alerts on Windows background?
- [ ] Write it up as REPORT Part 10 (setup, isolation, method, results table, what is not shown), then one row in `docs/RESULTS.md` and one line in README.
- [ ] Never commit samples, and keep the pcaps out of git (they stay in `downloads/`, which is ignored). Publish numbers, hashes and IOCs only.
- [ ] Be ready to explain the isolation (NAT only, no shared folders/clipboard, snapshots) and why you removed Defender inside the VM.

Phase 8 — Demo and interview prep (day 21 onward)

- [ ] **Record a 60–90 s demo** and put it at the top of the README: the closed-loop test or a lab-VM run where the dashboard shows the alert → block → drop counters. Tools: `asciinema` + `agg` (terminal GIF), or OBS / Peek (screen).
- [ ] **2-minute pitch:** problem (encrypted C2), approach (metadata, no decryption), architecture (3 planes), one result, one honest limit, what you'd do next.
- [ ] **One hard-bug story** (situation, problem, how you found it, fix, test). Good picks from RESULTS.md: the silent flow loss (ZeroMQ high-water mark), the 584,000-year flow duration (unsigned underflow on out-of-order timestamps), or the garbage JA4 (C union).
- [ ] **Fresh-install test:** clone the public repo into a new Ubuntu VM and follow only the README Quick start. Fix anything that breaks.
- [ ] **Mock interview:** ask someone (or Claude, with the code closed) to quiz you with the questions in this file. Answer without looking.

How to describe it on your CV

Good (true, specific):

- "Built an encrypted-traffic C2 detector: C sensor (libpcap, nDPI, JA4) → LightGBM scored with ONNX Runtime → automatic nftables blocking. Detected 5/5 unseen infections at 99.5 % precision."
- "Designed leakage-free evaluation (capture-grouped cross-validation, pre-labeled holdout) and measured real-host false-alarm rates."

Avoid until it's true: "kernel-level/eBPF blocking" (until Phase 6 is done), "production-ready", "real-time at line rate", "detects all RATs", "zero false positives".

Questions to expect (answers are in the files named)

1. Walk me through what happens to one packet, from the NIC to a block. (`main.c` → `daemon.py` → `mitigator.py`)
2. Why does this work on encrypted traffic? What can't it see?
3. What is JA4, and can malware fake it? (yes: and that's why it's one feature among 66)
4. How did you avoid data leakage? (grouped CV, holdout labeled first)
5. Your fold recall varies from 0.53 to 0.93. Why? (`evaluation.json`)
6. What is your false-positive rate in practice? (1 % per flow in CV; 9–67 blocks/hour on real hosts: and what you'd do about it)
7. How do you stop the system from blocking Google or your own DNS server? (`BlockPolicy.why_protected` , `_dns_server` , allowlist)
8. Could an attacker make you block something important? (forged scans; `_note_server`)

9. Why LightGBM and not a neural network?
10. Why C for the sensor? Why not do everything in eBPF?
11. How fast is it, and where is the bottleneck? (0.40 Mpps offline single thread; live not measured)
12. What was the hardest bug? (your story)
13. What did the AI write and what did you decide? Be ready with specific decisions you made and checked yourself: what to measure, which rule to keep, what not to claim.
14. What would you do next with 3 months?

Mastery checklist (tick only when you can do it without notes)

- [ ] Draw the architecture and name the format and transport on every arrow
- [ ] Explain all 23 sensor fields and the 66 model features by group
- [ ] Explain Welford, flow keys, idle vs active timeout
- [ ] Explain nDPI risk flags and JA4
- [ ] Explain grouped CV, calibration, and the base-rate problem with numbers
- [ ] Explain the block policy, the safety rules and the forged-scan defense
- [ ] Read the nftables ruleset aloud and explain each line
- [ ] Fix and attach the XDP program; explain why it failed before
- [ ] Rebuild and pass all tests from a fresh clone
- [ ] Give the honest limits in 30 seconds